Cyber Control and Game Learning

From continuous dynamics to sampled feedback, reinforcement learning, and multi-agent decisions

Lu-Xing Yang

This note explains how a cyber propagation model becomes a learning environment without losing its state, timing, intervention and cost semantics. A complete defender-learning example introduces Double DQN, paired policy comparisons and executed-action plots. Classical control, actor–critic methods, community control and adversarial games then follow from that example. The mathematical background and code-reading route are included, with explicit distinctions between implemented examples and further modeling choices. Prerequisites are elementary differential equations, probability, linear algebra and Python.

How to use this note

Read the model and the mathematical derivation before changing the code. Run the first example, inspect its state and decision curves, and then use the worked extensions. The equations, assumptions and interpretation are included here; the repository supplies the executable implementations.

Contents

1 From a dynamical model to a learned decision rule 3
1.1 The cyber-control research behind the examples . . . 3
1.2 Run the first example . . . 3
1.3 A common notation . . . 4

2 Start from interpretable cyber dynamics 4
2.1 The aggregate model used by the first experiment . . . 4
2.2 Costs used in the executable environment . . . 4
2.3 Two additional model templates . . . 5

3 Timing gives the environment its meaning 5
3.1 Flow actions, modes, intensities and impulses . . . 5
3.2 Use a state that contains the needed information . . . 6
3.3 Preserve the continuous objective in the reward . . . 6

4 The classical optimal-control connection 6
4.1 PMP supplies an open-loop benchmark . . . 6
4.2 What changes when learning is used? . . . 7
4.3 Three useful ways to reuse theory . . . 7

5 Value functions, Bellman equations and Double DQN 7
5.1 What is learned? . . . 7
5.2 Double DQN separates selection and evaluation . . . 8
5.3 Read the actual training configuration . . . 8
5.4 Termination is part of the objective . . . 9

6 Read the learning result as a control result 9
6.1 The default output files . . . 9
6.2 Separate three learning questions . . . 9
6.3 Keep comparisons physically matched . . . 10

7 Continuous actions and actor–critic methods 10
7.1 Why an actor and a critic? . . . 10
7.2 PPO and advantage estimation . . . 10
7.3 Deterministic actors and entropy-regularized actors . . . 10

8 Several agents: cooperation, competition and equilibrium 11
8.1 A Markov game retains separate rewards . . . 11
8.2 Centralized training and decentralized execution . . . 11
8.3 Train responses, then examine interaction . . . 11
8.4 Parameterized and impulsive game actions . . . 12

9 Retain network structure and operational constraints 12
9.1 Node-level dynamics and the incoming matrix . . . 12
9.2 Degree aggregation remains indexed by degree . . . 12
9.3 Enforce the resource constraint on executed actions . . . 13

10 From published control models to learning experiments 13
10.1 Preserve the decision problem during the conversion . . . 13
10.2 Parameter learning and MPC: an established feedback connection . . . 13
10.3 Impulse decisions and hybrid propaganda competition . . . 13
10.4 Awareness creates a state, an observation and an intervention question . . . 14

11 Read and adapt the code without adding more entry points 14
11.1 The small set of files to read first . . . 14
11.2 Adapt one physical mechanism at a time . . . 15
11.3 Exercises with worked answers . . . 15

A Neural and numerical essentials 15

B Connect this note to estimation and physics-informed learning 16

C Development commands 16

1 From a dynamical model to a learned decision rule

A propagation model explains how a cyber state changes under intervention. Optimal control specifies which interventions are desirable. Reinforcement learning supplies a way to learn decisions through repeated interaction with a model or environment. The connection is clearest when the same states, timing, costs and constraints are carried through all three descriptions.

The first experiment in this note is a defender facing a scripted attacker. It introduces the environment and Double DQN without requiring a network-sized state vector. Later sections retain individual nodes and introduce cooperation or competition. The aggregate example, the degree-class model in the foundation note, and the node-level extensions are distinct modeling resolutions.

1.1 The cyber-control research behind the examples

The research context begins with dynamical security evaluation under APTs and proceeds to strategic patching, repair, backup and recovery [15, 16, 17, 18]. Conflicting-opinion and rumor-refutation models supply social-network examples with competing objectives [19, 20]. Impulsive artificial defense, outsourcing and hybrid propaganda control motivate the explicit separation between flow actions and reset events [21, 22, 24]. Awareness-dependent propaganda suppression and coupled malware–opinion control add behavior and multiple network layers [25, 26].

These publications motivate the physical model and strategic questions. The first DDQN exercise below is a compact sampled SIR teaching environment, not a reproduction of every cited paper. Jafar et al.'s feedback-loop study provides a particularly direct learning–control bridge: reinforcement learning estimates unknown model parameters for an MPC-based malware-control loop [23]. That estimator role differs from the direct action-value learning used in the introductory DDQN. Both fit within an observation–estimation–decision–application loop when their interfaces are specified.

1.2 Run the first example

Inside `note1-cyber-control-games`, use a separate environment:

```
python -m venv .venv
# macOS / Linux:
source .venv/bin/activate
# Windows PowerShell instead:
# .venv\Scripts\Activate.ps1
python -m pip install -e .
python -m cybergames
```

Use Python 3.11 or 3.12 and have Git installed: installation obtains the tested shared `cybercontrol` dependency. The default runs on a CPU with generated initial conditions. It needs neither external data nor LaTeX. On a Windows/Linux CPU machine, PyTorch can be installed from its CPU wheel index before installing this package; follow the repository README for that installation option.

The command trains a small DDQN defender for 120 episodes with up to 30 decisions per episode, then evaluates it on ten held-out initial conditions. It compares five fixed actions, the untrained network with the same initialization, and the trained network. The purpose is to see how learning changes *executed decisions* and their physical consequences. The measured comparison, rather than the existence of a neural network, determines the result.

1.3 A common notation

Symbol	Meaning
$x(t)$	Continuous physical or cyber state.
t_r, Δ_r	Decision time and the duration to the next decision.
τ_ℓ	A physical intervention/event time, possibly distinct from t_r.
s_r, o_i^r	Markov state and the observation available to player i.
$a_r, u(t)$	Sampled action and its induced continuous intervention.
G, K	Reset map and instantaneous cost.
L, Φ	Running and terminal costs in a control problem.
π, Q, V	Policy, action value and state value.
γ_d	Learning discount factor; γ without a subscript is a recovery rate.

2 Start from interpretable cyber dynamics

2.1 The aggregate model used by the first experiment

Let S, I, R be susceptible, infected and recovered/protected fractions, with $S + I + R = 1$. The sampled actions select rates that are held during one decision interval:

$$\begin{aligned} \dot{S} &= -\beta_{\text{eff}} SI - pS + \omega R, \\ \dot{I} &= \beta_{\text{eff}} SI - (\gamma + c)I, \\ \dot{R} &= pS + (\gamma + c)I - \omega R. \end{aligned} \tag{1}$$

Patching p protects susceptible devices; cleaning c recovers infected devices. Deception reduces the effective infection rate during the interval. Protection wanes at rate ω. Each transition appears with opposite signs in its source and destination compartments, so mass is conserved.

The environment has five defender modes: no intervention, patch, clean, deceive and isolate. An integer mode uses unit intensity. A parameterized action additionally specifies $v \in [0, 1]$. In the current aggregate environment,

$$p = 0.30v\mathbf{1}_{\text{patch}}, \quad c = 0.25v\, s_A \mathbf{1}_{\text{clean}}, \quad h = 0.35v\mathbf{1}_{\text{deceive}}, \tag{2}$$

where $s_A = 0.5$ under a stealth attack and $s_A = 1$ otherwise. The transmission rate is

$$\beta_{\text{eff}} = \beta_A \max(0, 1 - \chi h). \tag{3}$$

The attacker selects scan, exploit, lateral or stealth. Their unit-intensity rate multipliers are respectively 1.10, 1.50, 1.70 and 1.20 times β_0. The teaching defaults are $\beta_0 = 0.65$, $\gamma = 0.05$, $\omega = 0.01$ and $\chi = 0.70$. They define an illustrative simulator, not an empirically fitted incident-response model.

Isolation is a reset, not an extra rate in (1):

$$S^+ = S^-, \qquad I^+ = (1 - 0.40v)I^-, \qquad R^+ = R^- + 0.40vI^-. \tag{4}$$

An alternative model may use continuous isolation, but its equations and cost must then be changed together. Calling both mechanisms “isolation” does not make them mathematically identical.

2.2 Costs used in the executable environment

Let $\bar{I}_r$ and $\bar{S}_r$ be interval averages obtained from the integrated trajectory. The defender’s interval reward is the negative of

$$C_D^r = \Delta_r \left[10\bar{I}_r + 0.5\bar{S}_r + p^2 + c_{\text{cmd}}^2 + 1.5h^2\right] + K_D^r, \tag{5}$$

where $c_{\text{cmd}} = 0.25v\mathbf{1}_{\text{clean}}$ is the commanded cleaning rate: the stealth reduction changes efficacy, not the charged commanded effort. When isolation is selected,

$$K_D^r = 2v^2 + 2.5(I^- - I^+); \tag{6}$$

otherwise $K_D^r = 0$. The impulse cost is charged once. It is not multiplied by the interval length or by the number of ODE substeps.

The environment also returns an attacker reward $\Delta_r[8\bar{I}_r + \bar{S}_r - c_A]$, where the action costs are $(0.05, 0.20, 0.30, 0.25)v_A^2$ in the listed attacker-mode order. These separate rewards are not exact negatives, so the environment is general-sum when both sides learn. The introductory DDQN experiment keeps the attacker scripted.

2.3 Two additional model templates

An APT model can separate compromise, detection and repair:

$$\begin{aligned} \dot{V} &= -\beta VC - u_p V + \omega P, & \dot{C} &= \beta VC - (\alpha + u_m)C, \\ \dot{D} &= (\alpha + u_m)C - u_r D, & \dot{P} &= u_p V + u_r D - \omega P. \end{aligned} \tag{7}$$

Here V, C, D, P denote vulnerable, covertly compromised, detected and protected states. Monitoring u_m and remediation u_r operate on different transitions. This template explains how to adapt the state and action mapping; it is not the three-state first experiment with renamed labels.

A misinformation model can instead track unaware users U, misinformation spreaders M, counter-message spreaders C and inactive users F:

$$\begin{aligned} \dot{U} &= -\beta_m UM - \beta_c UC + \delta F, & \dot{M} &= \beta_m UM - \kappa MC - \eta_m M, \\ \dot{C} &= \beta_c UC + \kappa MC - \eta_c C, & \dot{F} &= \eta_m M + \eta_c C - \delta F. \end{aligned} \tag{8}$$

A counter-messaging action might increase β_c or κ. A sudden campaign may instead define a reset. The learning environment should inherit the selected mechanism exactly.

3 Timing gives the environment its meaning

3.1 Flow actions, modes, intensities and impulses

A real-valued action is not the same as continuous-time feedback. An agent may select a real number only every hour and hold it constant between decisions. A discrete mode may choose which vector field operates without changing the state instantaneously. A mode–intensity pair (m, v) is a parameterized action; it becomes impulsive only when its map includes a reset.

For zero-order hold,

$$u(t) = \Gamma(a_r), \qquad t \in [t_r, t_{r+1}). \tag{9}$$

With a reset, a complete transition is

$$x_r^+ = G(x_r^-, a_r), \qquad x_{r+1}^- = \varphi_{\Delta_r}(x_r^+, \Gamma(a_r)), \tag{10}$$

where φ is the ODE flow. Without an impulse, G is the identity. If a physical event occurs inside the interval, integration must stop at that event, apply its reset and resume before the next policy observation.



In `SampledFlowImpulseEnv.step`, the order is pre-jump observation, joint action decoding, optional isolation reset, held-rate ODE integration and interval reward. The default $\Delta = 1$ and ten RK4 substeps describe two different grids: one decision interval contains ten numerical integration steps, but contributes only one replay transition. More substeps improve numerical accuracy; more decision epochs change the control problem.

3.2 Use a state that contains the needed information

The current observation is

$$s_r = [S_r, I_r, R_r, r/K]. \tag{11}$$

The fourth component is decision phase. It matters because the horizon is finite and the scripted attacker changes mode according to the decision index. The script uses scan for indices below 20, exploit from 20 through 59, and lateral movement thereafter. The default 30-decision example therefore encounters the first two regimes.

For a fixed horizon and known script, phase supplies that timing information. A model with a remaining budget, a hidden attacker mode or delays may require additional state. A partial observation is not automatically Markov. A history window, recurrent state or estimator is a possible approximation to missing information, and should be assessed as such rather than assumed to restore the Markov property exactly.

3.3 Preserve the continuous objective in the reward

For an undiscounted finite-horizon cost, the exact interval reward is

$$r_r = -K_r - \int_{t_r}^{t_{r+1}} L(x(t), u(t), t)\,\mathrm{d}t. \tag{12}$$

Include $-\Phi(x(T))$ once at termination, or equivalently use it as the terminal value. With $\gamma_d = 1$, maximizing the sum of these rewards is equivalent to minimizing the stated sampled-control objective. Setting γ_d merely "close to one" generally changes that objective.

For a continuously discounted cost with rate $\rho \geq 0$, a relative-time Bellman formulation uses

$$\begin{aligned} r_r &= -K_r - \int_0^{\Delta_r} e^{-\rho s} L(x(t_r + s), u(t_r + s), t_r + s)\,\mathrm{d}s, \\ \gamma_r &= e^{-\rho \Delta_r}. \end{aligned} \tag{13}$$

The products of interval discounts then produce the absolute factor $e^{-\rho t}$. Discounting only the next value while leaving the within-interval cost undiscounted is an approximation when $\rho > 0$. A variable interval must therefore affect reward integration and discounting consistently.

The environment approximates interval averages using composite Simpson quadrature for an even number of RK4 substeps, and trapezoidal quadrature otherwise. This is more accurate than charging damage only at the interval endpoint. The introductory DDQN uses $\gamma_d = 1$, matching its reported undiscounted finite-episode cost.

4 The classical optimal-control connection

4.1 PMP supplies an open-loop benchmark

For $\dot{x} = f(x, u, t)$, $x(0) = x_0$ and cost $J = \Phi(x(T)) + \int_0^T L(x, u, t)\,\mathrm{d}t$, define $H = L + \lambda^\top f$. Normal PMP conditions are

$$\dot{x} = f, \qquad \dot{\lambda} = -H_x, \qquad \lambda(T) = \Phi_x(x(T)), \qquad u(t) \in \arg\min_{v \in U} H(x(t), v, \lambda(t), t). \tag{14}$$

The state starts at 0 and the costate at T. The forward–backward sweep iterates those equations with a control update [1, 2]. It returns a schedule for the specified initial state and parameters, subject to numerical convergence and the necessary-versus-sufficient distinction.

For (15) below, use both patching and cleaning:

$$\begin{aligned} \dot{S} &= -\beta SI - u_p S + \omega R, \\ \dot{I} &= \beta SI - (\gamma + u_c) I, \\ \dot{R} &= u_p S + (\gamma + u_c) I - \omega R. \end{aligned} \tag{15}$$

With $L = qI + (b_p u_p^2 + b_c u_c^2)/2$ and terminal cost $q_T I(T)$,

$$\begin{aligned} \dot{\lambda}_S &= \beta I(\lambda_S - \lambda_I) + u_p(\lambda_S - \lambda_R), \\ \dot{\lambda}_I &= -q + \beta S(\lambda_S - \lambda_I) + (\gamma + u_c)(\lambda_I - \lambda_R), \\ \dot{\lambda}_R &= \omega(\lambda_R - \lambda_S), \qquad \lambda(T) = (0, q_T, 0)^\top. \end{aligned} \tag{16}$$

The bounded control updates are

$$u_p = \operatorname{clip}\left(\frac{S(\lambda_S - \lambda_R)}{b_p}, 0, \bar{u}_p\right), \qquad u_c = \operatorname{clip}\left(\frac{I(\lambda_I - \lambda_R)}{b_c}, 0, \bar{u}_c\right). \tag{17}$$

These equations show exactly which mechanism a demonstration control encodes. An impulse additionally changes costates through $\lambda^- = G_x^\top \lambda^+ + K_x$; it cannot be treated as an ordinary flow update without changing the problem.

4.2 What changes when learning is used?

PMP describes a continuous-time trajectory problem. A learned policy $a_r = \pi(s_r)$ describes a sampled feedback problem. Choosing a decision grid, restricting action modes, adding measurement noise or using a different objective may change the admissible problem. Therefore a cost comparison with an open-loop schedule should identify whether it measures matching-model optimization, feedback adaptation, robustness to mismatch, or a different operational action set.

A receding-horizon controller also reacts to observations by repeatedly solving a model-based problem. Feedback is not exclusive to RL. When feedback adaptation is the claimed benefit, a meaningful comparison may include a suitably designed feedback or MPC baseline as well as open-loop FBSM.

4.3 Three useful ways to reuse theory

Classical control can supply a baseline, demonstrations, or a residual-based learning objective. These uses are different. A reward-based learner uses the simulator and its objective; it does not solve the costate boundary-value problem merely because PMP was derived elsewhere. A demonstration-initialized actor uses additional teacher information. A PMP-informed neural method explicitly penalizes state, costate, boundary and constrained Hamiltonian conditions; Note 2 develops that formulation.

A constructive warm-start workflow generates control demonstrations over several initial states, maps them to the learner's actual admissible actions, trains the actor to imitate them, and then evaluates any subsequent reward-based improvement. Teacher information, approximation in the action map, and total training effort remain part of the method description.

5 Value functions, Bellman equations and Double DQN

5.1 What is learned?

For a policy π, the action value $Q^\pi(s, a)$ is the expected future discounted reward after taking a at s and then following π. A finite-horizon value also depends on remaining time, either explicitly or through a

phase component in s [3]. For a nonterminal transition,

$$Q^*(s,a) = \mathbb{E}\left[r + \gamma_d \max_{a'} Q^*(s',a') \mid s,a\right]. \tag{18}$$

At a true terminal state with terminal costs already included in the reward, there is no further value to bootstrap. For sampled transitions with terminal flag $d \in \{0,1\}$,

$$y = r + \gamma_d(1-d)\max_{a'} Q^-(s',a'). \tag{19}$$

The mask must be consistent with the episode definition. An artificial training interruption is not necessarily a terminal state of the physical decision problem.

5.2 Double DQN separates selection and evaluation

DQN approximates the action values by a neural network [4]. Double DQN uses the online network to select the next action and a target network to evaluate it [5]:

$$a' = \arg\max_b Q_\theta(s',b), \qquad y = r + \gamma_d(1-d)Q_{\theta^-}(s',a'). \tag{20}$$

The online network is fitted to y using replayed transitions. The implementation uses a smooth-L1 loss, a form of Huber loss, and periodically copies the online parameters to the target network. The target is computed without tracking gradients.

Listing 1: The learning update in mathematical order

```
# states, actions, rewards, next_states, terminals come from replay.
predicted = q(states).gather(1, actions)
with torch.no_grad():
    next_actions = q(next_states).argmax(1, keepdim=True)
    next_values = target(next_states).gather(1, next_actions)
    expected = rewards + gamma * (1.0 - terminals) * next_values
loss = torch.nn.functional.smooth_l1_loss(predicted, expected)
optimizer.zero_grad()
loss.backward()
torch.nn.utils.clip_grad_norm_(q.parameters(), 5.0)
optimizer.step()
```

Replay contains (s,a,r,s',d) for decision transitions. Random minibatches reuse previous interactions; they are not new environment interactions. The ϵ-greedy behavior rule explores with probability ϵ and otherwise chooses the largest online Q value. Greedy evaluation freezes the network and disables exploratory action sampling.

5.3 Read the actual training configuration

The introductory network takes the four entries in (11), has two hidden layers of width 32, and produces five Q values. These outputs are action values, not probabilities. The example uses Adam, a target update every 64 decision steps, and a batch size of $\min(32, K)$. Replay stores reward divided by ten to moderate Q magnitudes; reported returns and costs use the original reward units. Uniform positive reward scaling leaves the exact maximizing policy unchanged, while it can affect numerical training.

`ddqn.train` randomizes initial infection/protection fractions. Validation uses a separate seed. `cli.run_example` creates an untrained network with the same seed, width and depth as the training initialization, and freezes it. The final comparison then evaluates both networks and the five fixed-action policies on the same held-out initial conditions.

Listing 2: A small training call using the repository API

```
from cybergames.configs import DDQNConfig
from cybergames.ddqn import train, evaluate

cfg = DDQNConfig(
    episodes=20, horizon=30, eval_horizon=30,
    hidden=32, batch_size=32, gamma=1.0,
    log_every=5, device="cpu", seed=0
)
q_network, history = train(cfg)
score = evaluate(q_network, episodes=5, seed=10000, horizon=30)
print(score)
```

This shorter call demonstrates the interface, not a recommended performance budget. The normal CLI also generates paired baselines and figures.

5.4 Termination is part of the objective

The aggregate environment ends when its decision horizon is reached, when $I < 10^{-5}$, or when $I > 0.95$. Therefore its reported cost is the cost over that terminated episode. An application requiring continued maintenance after containment, or a penalty for catastrophic compromise, should encode that continuation/terminal cost explicitly. Otherwise stopping can remove future costs and change which policy looks attractive. Policy comparisons must share the same termination rule.

6 Read the learning result as a control result

6.1 The default output files

The `results/` directory contains `learning.png`, `infection.png`, `actions.png`, `training.csv`, `comparison.csv`, and `trajectories.csv`. Learning curves use validation return. The infection and action plots show the first held-out initial condition. The comparison table retains every evaluated case, so the printed averages can be traced back to paired outcomes.

The five fixed policies repeatedly choose no intervention, patch, clean, deceive or isolate. They are not fitted to the test cases. The comparison table uses cost, where lower is better; the validation curve uses reward, where higher is better. More intervention can reduce infected time while increasing total cost. Inspect both rather than ranking policies by infection alone.

6.2 Separate three learning questions

Parameter changes show that optimization occurred. Changed executed actions show that the policy changed. Improved held-out cost shows that the change was useful under the specified objective. These are three different statements. A decreasing temporal-difference loss is not sufficient evidence for the third.

A strong constant policy can be a genuine feature of a simple simulator. Retaining it helps students see when adaptive action selection matters. To study adaptive allocation, introduce a motivated source of variation—heterogeneous node risks, shared resources, changing observations or competing actions—and retain informative baselines under the resulting model.

For a new research variant, begin with a bounded small experiment. Compare an untrained policy, the trained policy, fixed actions and an appropriate rule/control-based method on common cases. Expand network size or interaction count after the action mapping and learning behavior are understood. Report uncertainty over independent training seeds separately from variation over test cases.

6.3 Keep comparisons physically matched

Common initial states, disturbances, graph samples, budgets, action bounds and evaluation horizons should be used where pairing is meaningful. When two methods receive different observations or training information, describe those differences. A teacher-initialized policy may have a useful learning advantage, but that is a different mechanism from extra information being available only at deployment.

Changing the number of episodes changes learning effort. Changing the horizon, decision interval or action costs changes the control problem. Changing numerical substeps changes integration accuracy. These changes answer different questions and should not be pooled into a single unnamed improvement.

7 Continuous actions and actor–critic methods

7.1 Why an actor and a critic?

A Q network with one output per discrete action is natural for five modes. A large continuous action vector cannot be enumerated so easily. An actor represents the policy, while a critic estimates its value or action value. A bounded scalar effort can be parameterized by

$$u = \underline{u} + (\bar{u} - \underline{u})\sigma(z), \qquad \sigma(z) = \frac{1}{1 + e^{-z}}. \tag{21}$$

With finite logits this lies strictly inside the interval; exact boundary actions require another representation or an explicit limiting/projection interpretation. Multiple independent sigmoid outputs do not enforce a shared budget.

7.2 PPO and advantage estimation

For a stochastic actor, define the likelihood ratio

$$\rho_r(\theta) = \frac{\pi_\theta(a_r \mid o_r)}{\pi_{\text{old}}(a_r \mid o_r)}. \tag{22}$$

PPO maximizes a clipped surrogate [6]:

$$\mathbb{E}\left[\min\left(\rho_r \widehat{A}_r, \text{clip}(\rho_r, 1-\epsilon, 1+\epsilon)\widehat{A}_r\right)\right]. \tag{23}$$

The advantage estimates whether an action performed better than the critic expected. A generalized advantage recursion uses

$$\begin{aligned} \delta_r &= r_r + \gamma_d(1 - d_r)V(s_{r+1}) - V(s_r), \\ \widehat{A}_r &= \delta_r + \gamma_d \lambda_{\text{GAE}}(1 - d_r)\widehat{A}_{r+1}. \end{aligned} \tag{24}$$

A rollout cut before a true terminal state needs the appropriate bootstrap value. In recurrent or partially observed settings, the actor state and stored information must also be consistent.

For a mode–intensity policy, specify its joint probability, for example $\pi(m, v \mid o) = \pi_m(m \mid o)\pi_v(v \mid o, m)$. The policy ratio must correspond to the distribution that actually generated the executed action. Clipping or repairing an action after sampling without accounting for its changed distribution can invalidate that ratio.

7.3 Deterministic actors and entropy-regularized actors

DDPG uses a deterministic actor with an action-value critic; SAC uses a stochastic actor and an entropy-regularized objective [7, 8]. The latter balances expected reward against policy entropy. These choices affect the optimized criterion and exploration, so an entropy term should not be confused with a physical intervention cost.

These methods provide background and extension routes. The primary packaged learning implementations are DDQN, a compact centralized-critic example, community MAPPO and adversarial self-play. The default command does not silently run every algorithm discussed in the theory.

8 Several agents: cooperation, competition and equilibrium

8.1 A Markov game retains separate rewards

A Markov game consists of a state space, one action/observation space per player, a joint-action transition law, and player-specific rewards [9]. In a cooperative problem, agents optimize a shared objective or a stated team criterion. In a zero-sum problem, one player's payoff is the negative of the other's. In a general-sum problem, each has its own tradeoffs.

A payoff-maximizing feedback Nash profile satisfies

$$V_i^{\pi_i^*, \pi_{-i}^*}(s) \geq V_i^{\pi_i, \pi_{-i}^*}(s) \tag{25}$$

for every admissible unilateral policy and every relevant state/time in the stated solution concept. An open-loop Nash schedule instead concerns unilateral time functions for a fixed initial condition. A Stackelberg problem additionally specifies commitment and response. These are distinct strategic questions [10].

8.2 Centralized training and decentralized execution

During training, a critic may use the full simulator state and joint actions,

$$Q_i(s, a_1, \ldots, a_N). \tag{26}$$

At execution, the actor uses only its declared observation, $\pi_i(a_i \mid o_i)$. This is centralized training with decentralized execution (CTDE) [11]. It can improve value estimation but does not eliminate every source of nonstationarity. Global information must not leak into a supposedly local actor at evaluation.

MAPPO applies PPO-style updates to multi-agent actors, often using a centralized value critic [12]. The cooperative community-control implementation and an adversarial attacker–defender experiment should be introduced separately. A shared-reward cooperative success is not evidence of an adversarial equilibrium.

8.3 Train responses, then examine interaction

A useful progression learns a defender against specified opponents, learns a response with the defender frozen, and then explores alternating or population-based training. Opponent pools expose a policy to more than the latest opponent. Evaluate a cross-play matrix with frozen row and column policies rather than relying only on the self-play training reward.

For a learned profile, the true unilateral improvement is

$$\Delta_i = \sup_{\pi_i} V_i^{\pi_i, \pi_{-i}^*} - V_i^{\pi_i^*, \pi_{-i}^*}. \tag{27}$$

A retrained response or a finite pool finds an attainable improvement, hence a lower bound on Δ_i. A large improvement demonstrates exploitability. A small observed improvement is evidence about the tested response class and optimization budget; it is not a certified global upper bound on exploitability. Independent player residuals, cross-play and matched unilateral-response searches provide complementary information.

8.4 Parameterized and impulsive game actions

An attacker and defender may each choose a mode and intensity. Specify whether their actions are simultaneous, whether either observes the other's current action, and how joint resets are ordered. Two noncommuting reset maps produce different outcomes under different orders. A simultaneous model needs an explicitly defined joint reset, not an incidental order in Python.

Between events, the joint mode can select a flow field $\dot{x} = f_{m_A,m_D}(x, v_A, v_D, t)$. At a genuine impulse, apply $G(x, a_A, a_D)$ and the relevant instantaneous payoffs. Cooperative, zero-sum and general-sum reward definitions remain unchanged by the fact that the state has a jump.

9 Retain network structure and operational constraints

9.1 Node-level dynamics and the incoming matrix

Let S_i, I_i, P_i denote node-level state probabilities. An illustrative heterogeneous SIPS structure is

$$\begin{aligned}
\dot{S}_i &= -s_i S_i \sum_j A_{ij} b_j I_j - e_i^p u_i^p S_i + \omega_i P_i, \\
\dot{I}_i &= s_i S_i \sum_j A_{ij} b_j I_j - (\gamma_i + e_i^c u_i^c) I_i, \\
\dot{P}_i &= e_i^p u_i^p S_i + (\gamma_i + e_i^c u_i^c) I_i - \omega_i P_i.
\end{aligned} \tag{28}$$

Here s_i includes the acquisition-rate scale, b_j is relative infectivity, and e_i^p, e_i^c are intervention efficacies. The shared library supplies node-SIPS dynamics and sparse pressure operations; the learning environment defines which nodes or communities receive each action.

For external graphs, use a fixed node order and transpose the directed NetworkX adjacency to obtain $A_{ij} = j \to i$. Keep the same order for state arrays, community labels, risks and observations. A small example is

```
import networkx as nx
G = nx.read_graphml("data/network.graphml")
nodes = list(G.nodes())
A = nx.to_scipy_sparse_array(
    G, nodelist=nodes, weight="weight", format="csr"
).T.tocsr()
```

The listing is a matrix-construction example, not a claim that the default aggregate DDQN accepts a graph CLI option. To use it in learning, adapt `node_env.py` and the corresponding observation/action dimensions.

9.2 Degree aggregation remains indexed by degree

For an undirected degree model, calculate $N_k, P(k), \bar{k}$ from the full graph before defining x_k and u_k. Its pressure is $\Theta = \sum_k kP(k)x_k/\bar{k}$ under the uncorrelated closure. A community index, a node index and a degree class are different labels. Aggregating an observation over a community does not change the underlying state equation into a degree model.

Node-level expected infected population is $\sum_i I_i$, and infected-node time is $\int \sum_i I_i \, \mathrm{d}t$. These quantities have interpretable population/time units. If the simulator stores binary node states, its raw infected count is integer-valued; fractional mean-field expectations and observed integer counts should be labeled accordingly.

9.3 Enforce the resource constraint on executed actions

A budget may limit the number of active communities, total patching effort, or simultaneous isolation events. Its interpretation determines the admissible action space. For a discrete budget, a mask or structured selection rule can prevent impossible actions. A continuous simplex or budget-constrained map can allocate shared effort. Independently clipping each control only enforces independent bounds.

The actor, replay record and evaluation must agree on the action actually executed. Recording an unconstrained proposal while executing a different budget-repaired action changes the transition associated with the recorded action. For PPO-style policies, the probability/log-probability also needs to match the executed sampling procedure.

10 From published control models to learning experiments

10.1 Preserve the decision problem during the conversion

The patch-injection, repair and backup/recovery studies give concrete strategic cyber-control settings [16, 17, 18]. To construct a learning environment from one of them, copy its state meanings, physical transition rates, admissible decisions and objective first. Specify what is observed at each decision time. Then approximate continuous controls by the chosen action representation and integrate the original cost over each interval.

Changing a continuous-valued control into five discrete actions restricts the policy class. Adding feedback observations expands the information pattern relative to an open-loop schedule. Either change can alter the attainable value. The comparison should state whether it measures numerical solution quality, feedback adaptation, information acquisition or a different intervention design. This makes the role of the learned policy interpretable rather than relying on the word "learning" alone.

10.2 Parameter learning and MPC: an established feedback connection

Jafar et al. combine learning-based parameter estimation and MPC for IoT malware propagation [23]. Their learning component concerns unknown coefficients of the dynamical model; it should not be described as the same action-selection procedure as `ddqn.py`. The distinction is visible in the information flow:

$$\text{observations} \longrightarrow \text{parameter update} \longrightarrow \text{predictive control} \longrightarrow \text{new observations.}$$

A direct DDQN instead maps its observation to action values and chooses an action. A hybrid learning/control experiment can use either approach, but should expose which object is learned: parameters, a value function, an initialization, an estimator or a policy.

A useful extension exercise compares fixed-parameter MPC, parameter-updating MPC and an observation-matched learned policy. Keep the physical simulator, intervention limits and test initial states the same. Include an untrained or unupdated counterpart for the learned component. Improvement caused by updated parameters is an inference benefit that can improve decisions even when the optimizer itself is unchanged.

10.3 Impulse decisions and hybrid propaganda competition

Impulsive artificial defense and defense outsourcing motivate environments in which an action applies an immediate reset and incurs an event cost [21, 22]. Derive that reset from the chosen application; a rate adjustment held over a decision interval is a flow action. The two mechanisms have different transitions, costs and learning consequences.

The propaganda-war paper provides a degree-based two-player hybrid example [24]. Its code separates network preprocessing, forward propagation, backward adjoints and player-specific strategy updates. To study learned feedback on that model, construct observations and action maps for both players while

preserving their individual payoff definitions. Continuous effort and event intensity may be represented by different action components. Event times can remain fixed, coincide with decision times, or be selected by a policy, but the choice must be part of the environment definition.

The original forward–backward game and a new multi-agent learning formulation answer related but distinct solution questions. A cooperative MAPPO implementation is not automatically suitable for opposing objectives. Evaluate competitive policies against frozen opponents and alternative responses, and state whether the solution sought is open-loop Nash, feedback Nash, Stackelberg or a restricted-policy comparison.

10.4 Awareness creates a state, an observation and an intervention question

The awareness-based propaganda and coupled malware–opinion studies distinguish a propagation state from an awareness/opinion state [25, 26]. Their interventions can affect those states through different channels. A learning adaptation should therefore decide which state is observable, whether an estimate is supplied to the policy, and whether an action modifies infection directly or changes subsequent susceptibility through awareness.

Ablating awareness changes the mechanism or information available to the controller. Freezing a learned policy changes the decision rule. Disabling an impulse changes the admissible intervention. These comparisons should be labeled separately. The same principle applies to the conflicting-opinion and rumor-refutation games: each player's benefit/cost and the population interpretation must remain intact when introducing an actor and critic [19, 20].

The small node-level and multi-agent examples in this repository are teaching extensions. The papers and attributed research code supply the application-specific equations and constraints. The learning machinery here provides an explicit route to new experiments without presenting those extensions as already established results of the cited publications.

11 Read and adapt the code without adding more entry points

11.1 The small set of files to read first

File	Role in the learning sequence
`cli.py`	Complete first experiment: configure, train, compare frozen policies, save figures and tables.
`envs.py`	State, observation, action-to-rate map, reset, ODE transition and rewards.
`ddqn.py`	Online/target networks, replay, Double DQN target and policy evaluation.
`configs.py`	Explicit model and learning settings.

Read those files before the cooperative node-level or adversarial extensions. Shared numerical and neural building blocks are reused rather than copied into each example. The optional broader study is

```
python -m cybergames study --device cpu --output-dir results/study
```

It uses its own experiment profiles; the first example's `-episodes` flag does not redefine every method inside that preset.

11.2 Adapt one physical mechanism at a time

For a new model, specify the state and units, derive the transitions, and then revise the observation, action map, reward and termination rule. Match the policy's input dimension to the observation and its output to the admissible action representation. A new cost weight changes the decision problem; a new neural width changes its approximation.

For a new impulse, define its reset and instantaneous cost before coding. For a delayed state, retain the required history and use an appropriate delay solver. For a time-varying graph, advance the graph schedule in the same physical time units as the decisions. For a partial-observation model, assess the estimator or memory policy as part of the decision method.

Changing the mathematical model also changes any PMP teacher or baseline. Update its Jacobian, costate equations and control optimizer rather than importing a schedule from an incompatible mechanism. When inference is used upstream, carry parameter uncertainty into the control comparisons instead of treating a fitted rate as known truth.

11.3 Exercises with worked answers

Impulse cost. A reset costs 2 units and the next decision is 0.25 hours later. What is the impulse contribution to reward?

Answer: -2, not -0.5. Only the running rate cost is integrated over the quarter-hour interval.

Terminal target. A terminal transition has reward -3, discount 1 and a target network predicting future value 10. What is the correct Double DQN target?

Answer: -3, because $(1 - d) = 0$. Adding 10 would bootstrap past the defined episode.

Learning evidence. Network parameters change, but every greedy action remains patch. What has been demonstrated?

Answer: Optimization changed the function approximation. It has not yet demonstrated a change in the executed greedy policy or an improvement over repeated patching.

Comparison scope. A feedback policy beats an open-loop schedule after a new infection shock. Does that prove a better solution to the original deterministic control problem?

Answer: It provides evidence of adaptation under that shock. Pure optimization superiority requires matching the original dynamics, information and admissible decisions.

A Neural and numerical essentials

An MLP composes affine maps and nonlinear activations. A Q network outputs real action values. A categorical actor transforms logits into probabilities using softmax. A continuous actor maps outputs to an admissible effort space. A critic estimates a scalar state/action value. These roles should be clear before selecting width and depth.

RK4 evaluates four slopes for each numerical step. In a sampled environment, the action is held according to the specified action map during those evaluations. A projection can remove small numerical violations of a simplex, but repeated large projections alter the dynamics. Test step refinement and conservation at the equation level before using trajectories as learning data.

A basic NumPy/PyTorch model pair should agree on state ordering and rates. Seeded initial-condition pairing improves comparisons; it does not guarantee bitwise equality across all hardware. Computation

time, learning interactions and solver evaluations are different measures and should be reported separately when relevant.

B Connect this note to estimation and physics-informed learning

A neural ODE represents part or all of a vector field and differentiates through integration. A PINN represents trajectories and trains using equation residuals. PIDL is a broader model-and-data learning approach [13, 14]. These can estimate a mechanism before control, or produce equation-informed numerical candidates. They do not automatically choose operational actions or establish equilibrium.

A useful end-to-end chain is observation, state/parameter estimation, decision, physical application and new observation. The foundation note supplies continuous and impulsive control theory; Note 2 develops the inference and residual-learning components. Keeping their interfaces explicit makes it possible to identify whether an improvement comes from better state information, better modeling, better allocation or learned feedback.

C Development commands

```
python -m pip install -e ".[dev]"
pytest -q
python -m cybergames check
python -m cybergames docs
```

Only the final command requires LaTeX/`latexmk`. The default learning experiment remains `python -m cybergames`. Tests check implementation properties; performance conclusions come from the specified learning and policy comparisons.

References

[1] L. S. Pontryagin et al., *The Mathematical Theory of Optimal Processes*. Wiley, 1962.

[2] S. Lenhart and J. T. Workman, *Optimal Control Applied to Biological Models*. Chapman and Hall/CRC, 2007.

[3] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, second edition. MIT Press, 2018.

[4] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, 518, 529–533, 2015.

[5] H. van Hasselt, A. Guez and D. Silver, "Deep reinforcement learning with double Q-learning," *AAAI*, 2016.

[6] J. Schulman et al., "Proximal policy optimization algorithms," arXiv:1707.06347, 2017.

[7] T. P. Lillicrap et al., "Continuous control with deep reinforcement learning," arXiv:1509.02971, 2015.

[8] T. Haarnoja et al., "Soft actor-critic algorithms and applications," arXiv:1812.05905, 2018.

[9] M. L. Littman, "Markov games as a framework for multi-agent reinforcement learning," *International Conference on Machine Learning*, 157–163, 1994.

[10] T. Başar and G. J. Olsder, *Dynamic Noncooperative Game Theory*, second edition. SIAM, 1999.

[11] R. Lowe et al., "Multi-agent actor-critic for mixed cooperative-competitive environments," *Advances in Neural Information Processing Systems*, 2017.

[12] C. Yu et al., "The surprising effectiveness of PPO in cooperative multi-agent games," arXiv:2103.01955, 2021.

[13] R. T. Q. Chen, Y. Rubanova, J. Bettencourt and D. Duvenaud, "Neural ordinary differential equations," *Advances in Neural Information Processing Systems*, 2018.

[14] M. Raissi, P. Perdikaris and G. E. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations," *Journal of Computational Physics*, 378, 686–707, 2019.

[15] L.-X. Yang, P. Li, X. Yang and Y. Y. Tang, "Security evaluation of the cyber networks under advanced persistent threats," *IEEE Access*, 5, 20111–20123, 2017. doi:10.1109/ACCESS.2017.2757944.

[16] L.-X. Yang, P. Li, X. Yang, Y. Xiang and W. Zhou, "A differential game approach to patch injection," *IEEE Access*, 6, 58924–58938, 2018. doi:10.1109/ACCESS.2018.2874446.

[17] L.-X. Yang, P. Li, Y. Zhang, X. Yang, Y. Xiang and W. Zhou, "Effective repair strategy against advanced persistent threat: A differential game approach," *IEEE Transactions on Information Forensics and Security*, 14(7), 1713–1728, 2019. doi:10.1109/TIFS.2018.2885251.

[18] L.-X. Yang, K. Huang, X. Yang, Y. Zhang, Y. Xiang and Y. Y. Tang, "Defense against advanced persistent threat through data backup and recovery," *IEEE Transactions on Network Science and Engineering*, 8(3), 2001–2013, 2021. doi:10.1109/TNSE.2020.3040247.

[19] L.-X. Yang, P. Li, X. Yang, Y. Xiang and Y. Y. Tang, "Simultaneous benefit maximization of conflicting opinions: Modeling and analysis," *IEEE Systems Journal*, 14(2), 1623–1634, 2020. doi:10.1109/JSYST.2020.2964004.

[20] D.-W. Huang, L.-X. Yang, P. Li, X. Yang and Y. Y. Tang, "Developing cost-effective rumor-refuting strategy through game-theoretic approach," *IEEE Systems Journal*, 15(4), 5034–5045, 2021. doi:10.1109/JSYST.2020.3020078.

[21] H. Sun, X. Yang, L.-X. Yang, K. Huang and G. Li, "Impulsive artificial defense against advanced persistent threat," *IEEE Transactions on Information Forensics and Security*, 18, 3506–3516, 2023. doi:10.1109/TIFS.2023.3284564.

[22] Y. Qin, X. Yang, L.-X. Yang and K. Huang, "Modeling and study of defense outsourcing against advanced persistent threat through impulsive differential game approach," *Computers & Security*, 145, 104003, 2024. doi:10.1016/j.cose.2024.104003.

[23] M. T. Jafar, L.-X. Yang, G. Li, Q. Zhu and C. Gan, "Minimizing malware propagation in Internet of Things networks: An optimal control using feedback loop approach," *IEEE Transactions on Information Forensics and Security*, 19, 9682–9697, 2024. doi:10.1109/TIFS.2024.3463965.

[24] X. Cheng, L.-X. Yang, Q. Zhu, C. Gan, X. Yang and G. Li, "Cost-effective hybrid control strategies for dynamical propaganda war game," *IEEE Transactions on Information Forensics and Security*, 19, 9789–9802, 2024. doi:10.1109/TIFS.2024.3468903. Author code: PropagandaWar_TIFS_2024_Code.

[25] X. Cheng, L.-X. Yang, Q. Zhu, C. Gan and G. Li, "Impulse strategies for suppressing cyber propaganda with awareness," *IEEE Transactions on Computational Social Systems*, 12(5), 2685–2695, 2025. doi:10.1109/TCSS.2024.3522889. Author code: Propaganda_TCSS_2025_Code.

[26] X. Cheng, L.-X. Yang, G. Li, Z. Ma, T. Zhu, L. Wang and S. Duan, "Modeling and mitigating social engineering malware: Integrating malware-opinion dynamics with optimal impulse control approaches," *IEEE Transactions on Information Forensics and Security*, 20, 12231–12244, 2025. doi:10.1109/TIFS.2025.3630005. Author code: OpinionMalware_TIFS_2025_Code.